Practice · Comparisons and logic

Each question lists its options; the right one is marked **Right answer**. *Feedback* is what students see after they answer. In a typed question the blank is in square brackets, with the accepted answers separated by `|`.

(mc) L1 · Which one means "is equal to"

Which symbol asks "are these two values the same?"

- `=`
- **Right answer:** `==`
- `=>`
- `!=`

Feedback: Two equals signs, `==`, compare. One equals sign, `=`, puts a value into a box. Slide 9's warning: `=` assigns, `==` compares.

(mc) L1 · Which one means "not equal"

Which symbol asks "are these two values different?"

- `<>`
- `=/=`
- **Right answer:** `!=`
- `!==`

Feedback: `!=` is "not equal to". `5 != 3` is true, so it is 1. The `!` on its own means "not", and you see it again with `&&` and `||`.

(mc) L1 · What a comparison gives back

What is the value of `5 > 3` in C?

- `true`, a word.
- `yes`
- **Right answer:** `1`
- `5`

Feedback: C has no separate true and false words in this course. A comparison gives a number: **1** when it is true, **0** when it is false. `5 > 3` is true, so it is 1. `printf("%d", 5 > 3)` prints `1`.

(mc) L2 · Two comparisons, two numbers

```
int age = 19;
printf("%d\n", age >= 18);
printf("%d\n", age == 18);
```

What are the two lines printed?

- `true` and `false`
- `1` and `1`
- **Right answer:** `1` and `0`
- `19` and `18`

Feedback: `19 >= 18` is true, so 1. `19 == 18` is false, so 0. Each comparison becomes a number, and `%d` prints that number.

(mc) L2 · AND

```
int age = 20;
int hasCard = 1;
printf("%d\n", age >= 18 && hasCard);
```

What is printed?

- **Right answer:** `1`
- `0`
- `20`
- `21`

Feedback: `&&` is AND: both sides must be true. `age >= 18` is true (1). `hasCard` is 1, which counts as true. True and true is true, so the answer is 1. If either side were 0, the answer would be 0.

(mc) L2 · OR and NOT

```
int student = 0;
int age = 16;
printf("%d\n", student || age < 18);
printf("%d\n", !(age < 18));
```

What are the two lines printed?

- `0` and `1`
- `1` and `1`
- **Right answer:** `1` and `0`
- `0` and `0`

Feedback: `||` is OR: at least one side must be true. `student` is 0, but `age < 18` is true, so the OR is 1. `!` is NOT: it flips the answer. `age < 18` is 1, and `!` makes it 0. So `1` then `0`.

(fib) L3 · Type the "equal to" symbol

Complete the condition so it checks whether `score` is exactly 100. Type only the symbol.

```
if (score [==] 100) {
    printf("Perfect\n");
}
```

Feedback: `==` compares. If you type one `=`, the program compiles with a warning, puts 100 into `score`, and the condition is always true.

(fib) L3 · Type the AND symbol

Complete the condition so the person must be 18 or older **and** have a card. Type only the symbol.

```
if (age >= 18 [&&] hasCard) {
    printf("Allowed\n");
}
```

Feedback: `&&` is AND. Both parts must be true for the `if` block to run. One `&` alone means something else in C, so always type two.

(fib) L3 · Type the "not equal" symbol

Complete the condition so the message prints when `answer` is anything other than the letter `y`. Type only the symbol.

```
if (answer [!=] 'y') {
    printf("Stopping\n");
}
```

Feedback: `!=` is "not equal to". The block runs for every character except `'y'`. Note the single quotes around `y`: a `char` value.